Cache busting for UI and Micro-Frontends

Overview and why?

After every PROD release, if we have any changes to our UI and/or Micro-Frontends/Micro-UI(s) code. We are requiring a ***hard refresh*** or ***cache clearing*** from our end users (MDs). If the appropriate steps aren't performed from the end users, it can sometimes cause undue noise with regards to non-issues being reported or just not ideal user experience. So, we want to find ways in which we could eliminate the need for such actions to be performed by our end users.

Cache Busting

We employ caching of our static files so as increase performance of our application, so the files that don't change frequently aren't downloaded every time but are instead **cached**. But when they change, we should employ a process to bust the cache instead of relying on end users.

Approaches

- **Using webpack to create JS file name has a hash and implement no-cache on the html file which references it.**

This approach is already implemented for our micro-ui host ***UCX.UI_2***. Which means any updates made to the code for the host will be received to the end users without the need for cache clearing?

- **Manual updates to the micro-frontend static JS file URL in UCX.UI_2**

This approach relies on developers to update the URL with an additional ***querystring*** something like ***$version=02-20-2024*** in the configuration of our micro-ui host ***UCX.UI_2*** for the micro-ui they are updating.

Few teams are already doing this, here are some PR examples.

[Commit 84e54828: Merged PR 83526]
[Commit 491d3bd3: Merged PR 83291]

- **Automated updates to the micro-frontend static JS file URL in UCX.UI_2**

Currently the configuration/logic on where the files of a micro-frontend reside is coded within the micro-frontend host. The logic is enhanced to collate the static JS URL with a version which is obtained from another static source.

An implementation exists currently and here are some PRs for the same.

[UI_2 enhancement]

[Front-Door biceps microUI]

[Front-Door terraform microUI]

[template changes for json file updating].

```
microComponents: [
  {
    name: 'ucx-attachments',
    url: 'https://dv1ucxmicroui.blob.core.windows.net/ucxattachmentsmicroui/ucx-attachments.js?sp=r&st=2022-05-10T13:43:52Z&se=2027-05-10T21:43:52Z&spr=https&sv=2020-08-04&sr=b&sig=pZ2T5Rf0bGK0Ur5rDoyW2e39TWqLQPh0zPbiSzYrylM%3D&version=01-11-2024',
    version: 'https://dv1ucxmicroui.blob.core.windows.net/ucxattachmentsmicroui/version.json?sp=r&st=2024-02-20T17:29:34Z&se=2034-02-21T01:29:34Z&spr=https&sv=2022-11-02&sr=b&sig=HFR%2BcgxcagbbcsSMB7UGVCRJ17hz1d1NEVxrFcphMms%3D'
  },
  {
    name: 'ucx-request-provider-details',
    url: 'https://eheu2dv1ucxrpdetailfd.azurefd.net/ucx-request-provider-details.bundle.js?ts=2024-01-10T14:05:53Z',
    version: 'https://eheu2dv1ucxrpdetailfd.azurefd.net/version.json'
  }
];
```

So, using the following approach, UCX.UI_2 will not have to change if some micro-frondend's code changes, after the first change to add the version URL.